

Relatório da atividade.

O que eu fiz

Nesta atividade eu melhorei a interface do Faith Letters, o app de cartas para missionários. Usei CSS Modules para o estilo, criei componentes separados e apliquei memorização, Lazy Loading e Suspense.

Primeiro criei a estrutura da página no `App.tsx`, com header, main e footer, e o `App.module.css` para estilizar. Depois criei os cards `Missionary Card` e `Letter Card`, cada um com seu próprio CSS Module. Em seguida fiz as listas, `Missionary List` e `LetterList`. A lista de missionários mostra uma mensagem de carregamento enquanto os dados chegam, e a de cartas mostra uma mensagem amigável quando não há nenhuma carta. Por fim, otimizei a aplicação e criei os Skeletons.

CSS Modules

Usei CSS Modules para que o estilo de um componente não interfira no de outro. O React gera um nome único para cada classe, então posso ter uma classe `.card` no `Missionary Card` e outra no `Letter Card` sem conflito. Isso deixa o código mais organizado e fácil de reaproveitar.

Memorização

Usei `React.memo` nos dois cards. Com ele, o card só é redesenhado quando a prop muda. Se a lista renderizar de novo mas os dados do missionário forem os mesmos, o React pula aquele card.

Usei `useMemo` para calcular a bandeira do país. A função `getCountry Flag` só roda de novo quando o país muda. Reconheço que, com poucos cards, o ganho é pequeno. Mesmo assim, apliquei porque a atividade pede e porque é uma boa prática quando as listas crescerem. A atividade avisa para não otimizar cedo demais, então entendo que o ideal é medir antes de aplicar.

Lazy Loading

Carreguei `Missionary List` e `LetterList` com `React.lazy`. Sem isso, o navegador baixou o código de tudo logo no começo. Com o lazy, o código de cada lista vira um arquivo separado e só é baixado quando a lista aparece. Assim o bundle inicial fica menor e a primeira tela carrega mais rápido. Conferi isso na aba Rede do navegador, onde vi os arquivos das listas sendo carregados separadamente. Para funcionar, os componentes precisam usar `export default`.

Suspense e Skeletons

O `React.lazy` precisa do `Suspense` por perto. Enquanto o código da lista é baixado, o `Suspense` mostra o `fallback`. Em vez de uma tela em branco, usei Skeletons (`MissionaryListSkeleton` e `LetterList Skeleton`): blocos cinza com o mesmo formato dos cards, com uma animação de brilho. Isso melhora a percepção de velocidade e evita que o layout "pule" quando o conteúdo chega. Testei simulando uma internet lenta (3G) no navegador e vi os Skeletons aparecendo antes dos cards.

Conclusão

Com essas técnicas, a aplicação ficou mais organizada, com estilos isolados, componentes reaproveitáveis, carregamento mais eficiente e uma experiência melhor para o usuário. Entendi para que serve cada técnica e quando faz sentido usar.